

Assignment 4 DESIGN.pdf:

Description of Program:

This program provides a solution to the traveling salesman problem. It gets an input file from the user and creates a graph and finds the shortest hamiltonian path.

Files to be included in directory “asgn4” + pseudocode/structure:

- graph.c
 - implements the graph ADT.
 - typedef struct Graph Graph
 - given in pdf
 - Graph *graph_create(uint32_t vertices, bool undirected)
 - constructor for graph
 - Given in pdf
 - void graph_delete(Graph **G)
 - destructor for graph
 - Given in pdf
 - uint32_t graph_vertices(Graph *G)
 - return g.vertices
 - bool graph_add_edge(Graph *G, uint32_t i, uint32_t j, uint32_t k)
 - if vertices within bounds
 - G.matrix[i][j] = k
 - If undirected

- `G.matrix[j][i] = k`
 - return true
 - else return false
- `bool graph_has_edge(Graph *G, uint32_t i, uint32_t j)`
 - if vertices within bounds and `G.matrix[i][j]` is greater than 0
 - Return true
 - else return false
- `uint32_t graph_edge_weight(Graph *G, uint32_t i, uint32_t j)`
 - if graph has edge
 - Return weight
 - else return false
- `bool graph_visited(Graph *G, uint32_t v)`
 - If visited return true
 - Else return false
- `void graph_mark_visited(Graph *G, uint32_t v)`
 - If vertex within bounds
 - Set `g.visited` to true
- `void graph_mark_unvisited(Graph *G, uint32_t v)`
 - If vertex within bounds
 - Set `g.visited` to false
- `void graph_print(Graph *G)`
 - Print matrix using `PRlu32`
- `graph.h`

- specifies the interface to the graph ADT.
- typedef struct Graph Graph;
- Graph *graph_create(uint32_t vertices, bool undirected);
- void graph_delete(Graph **G);
- uint32_t graph_vertices(Graph *G);
- bool graph_add_edge(Graph *G, uint32_t i, uint32_t j, uint32_t k);
- bool graph_has_edge(Graph *G, uint32_t i, uint32_t j);
- uint32_t graph_edge_weight(Graph *G, uint32_t i, uint32_t j);
- bool graph_visited(Graph *G, uint32_t v);
- void graph_mark_visited(Graph *G, uint32_t v);
- void graph_mark_unvisited(Graph *G, uint32_t v);
- void graph_print(Graph *G);
- path.c
 - implements the path ADT
 - typedef struct Path Path
 - given in pdf
 - Path *path_create(void)
 - constructor for path
 - void path_delete(Path **p)
 - Destructor for path
 - bool path_push_vertex(Path *p, uint32_t v, Graph *G)
 - If stack full return false
 - Use peak to get last vertex

- Add graph edge weight to p length
 - Call stackpush and return if able to do it or not
- `bool path_pop_vertex(Path *p, uint32_t *v, Graph *G)`
 - If stack empty return false
 - Use stack pop
 - Use stack peak
 - Decrease p length by graph edge weight
 - Return true
- `uint32_t path_vertices(Path *p)`
 - return stacksize of pvertices
- `uint32_t path_length(Path *p)`
 - Return plength
- `void path_copy(Path *dst, Path *src)`
 - Copy contents of src to dst
 - Copy length of src to dst
- `void path_print(Path *p, FILE *outfile, char *cities[])`
 - Use stack print
- `path.h`
 - specifies the interface to the path ADT.
 - `typedef struct Path Path;`
 - `Path *path_create(void);`
 - `void path_delete(Path **p);`
 - `bool path_push_vertex(Path *p, uint32_t v, Graph *G);`

- `bool path_pop_vertex(Path *p, uint32_t *v, Graph *G);`
- `uint32_t path_vertices(Path *p);`
- `uint32_t path_length(Path *p);`
- `void path_copy(Path *dst, Path *src);`
- `void path_print(Path *p, FILE *outfile, char *cities[]);`
- `stack.c`
 - implements the stack ADT
 - `typedef struct Stack Stack`
 - Given in pdf
 - `Stack *stack_create(uint32_t capacity)`
 - Given in pdf
 - Constructor for stack
 - `void stack_delete(Stack **s)`
 - Given in pdf
 - Destructor for stack
 - `uint32_t stack_size(Stack *s)`
 - Return stop
 - `bool stack_empty(Stack *s)`
 - If stacksize = 0
 - Return true
 - Else
 - Return false
 - `bool stack_full(Stack *s)`

- Return true if stacksize = s.capacity
 - Else return false
- bool stack_push(Stack *s, uint32_t x)
 - If s is null or s is full
 - Return false
 - Else
 - S.items at top = x
 - S.top++
 - Return true
- bool stack_pop(Stack *s, uint32_t *x)
 - If s is empty
 - Return false
 - *x = s.items(top-1)
 - Decrement s.top
 - Return true
- bool stack_peek(Stack *s, uint32_t *x)
 - If stack empty
 - Return false
 - Else
 - *x = s.items(top-1)
 - Return true
- void stack_copy(Stack *dst, Stack *src)
 - if capacities are equal

- Set tops to equal
 - Copy every item from src to dst
- void stack_print(Stack *s, FILE *outfile, char *cities[])
 - Print stack
 - Given in pdf
- stack.h
 - specifies the interface to the stack ADT
 - typedef struct Stack Stack;
 - Stack *stack_create(uint32_t capacity);
 - void stack_delete(Stack **s);
 - uint32_t stack_size(Stack *s);
 - bool stack_empty(Stack *s);
 - bool stack_full(Stack *s);
 - bool stack_push(Stack *s, uint32_t x);
 - bool stack_pop(Stack *s, uint32_t *x);
 - bool stack_peek(Stack *s, uint32_t *x);
 - void stack_copy(Stack *dst, Stack *src);
 - void stack_print(Stack *s, FILE *outfile, char *cities[]);
- dfs.c
 - implements dfs.h
 - Followed pseudo code given in pdf
 - have counter to keep track of recursion calls
 - procedure DFS(G,v):

- label v as visited
- for all edges from v to w in G.adjacentEdges(v) do
 - if vertex w is not labeled as visited then
 - recursively call DFS(G,w)
- label v as unvisited

- dfs.h
 - specifies the interface for dfs
 - void dfs(Graph *G, uint32_t v, Path *curr, char *cities[], Path *shortest, int *counter, FILE *outfile, bool verbose);
- vertices.h
 - defines macros regarding vertices.
- tsp.c
 - contains main()
 - Parse command-line options with looped calls to getopt()
 - Scan in the first line from the input (number of vertices)
 - Print an error if the number specified is greater than VERTICES
 - read the next n lines from the input using fgets(), save line in cities array using strdup. if the line is malformed, print an error and exit the program
 - create new graph
 - scan the rest of input until the end of the file. Add edges accordingly. Print error if invalid format

- Starting from the origin vertex, defined by the macro `START_VERTEX` in `vertices.h`, perform a depth-first search on `G` to find the shortest Hamiltonian path.
- After the search, print out the length of the shortest path found, the path, and the number of calls to `dfs()`
- Corresponding usage:
 - `-h` : Help message. Prints the usage of the program
 - `-v` : Verbose Printing. Prints each new optimal path as it is found
 - `-u` : Uses an undirected graph
 - `-i infile` : specifies the input file
 - `-o outfile` : specifies the output file
- Makefile
 - `CC = clang` must be specified.
 - `CFLAGS = -Wall -Wextra -Werror -Wpedantic` must be specified.
 - `make` must build the `Mathlib-test` executable, as should `make all` and `make mathlib-test`
 - `make clean` must remove all files that are compiler generated.
 - `make format` should format all your source code, including the header files.
- README.md
 - In proper Markdown syntax.
 - Describe how to use your program and Makefile

- Lists and explains any command-line options that program accepts.
 - Reports any false positives by scan-build
 - Notes down any known bugs or errors in this file as well for the graders.
- DESIGN.pdf
 - This document must be a proper PDF.
 - Describes design and design process for program with enough detail such that a sufficiently knowledgeable programmer would be able to replicate implementation.